



7. Efficient Memory Management for Large Language Model Serving with PagedAttention

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work

3. 제안 방법론

Main Idea

Contribution

4. 실험 및 결과

Dataset

Baseline

결과

5. 결론 (배운점)

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

1. Introduction

논문이 다루는 분야

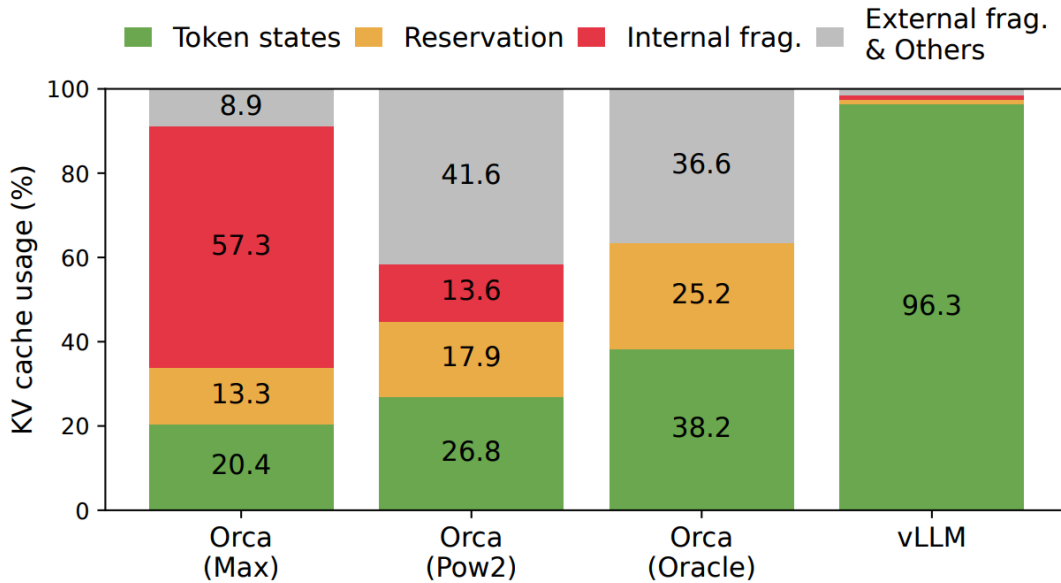
해당 task에서 기존 연구 한계점

논문의 contributions

1. Introduction

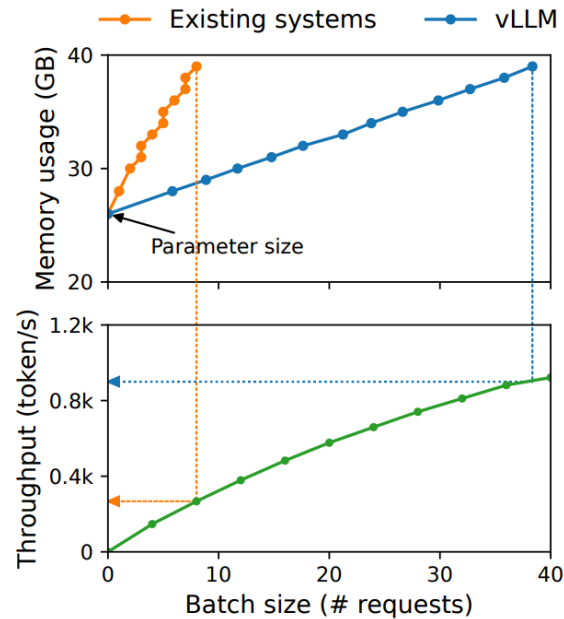
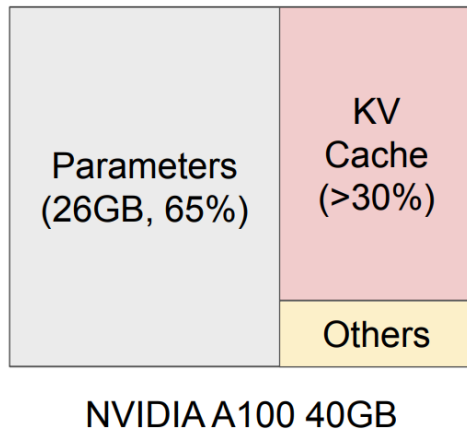
논문이 다루는 분야

- 대규모 언어 모델(LLM)의 높은 처리량(High throughput)을 달성하기 위한 서빙(Serving) 시스템 최적화 분야
- 특히 LLM 추론 시 발생하는 어텐션 키-밸류 캐시(KV Cache)의 메모리 관리 및 동적 할당 문제를 운영체제의 가상 메모리 페이징 기법을 차용해 해결하는 방법론



해당 task에서 기존 연구 한계점

- LLM의 자가 회귀(Autoregressive) 디코딩 특성상 입력과 출력의 길이를 미리 알 수 없어 KV 캐시 메모리가 동적으로 변하고 예측이 불가능함
- 기존 시스템(FasterTransformer, Orca 등)은 요청의 최대 길이에 맞춰 연속된 물리적 메모리 공간을 미리 정적 할당하므로, 실제 사용되지 않는 공간이 낭비되는 심각한 내부 단편화(Internal fragmentation)가 발생함
- 또한 연속적인 메모리 할당 방식으로 인해 외부 단편화(External fragmentation) 문제도 발생하여 전체 메모리 공간의 비효율성을 초래함
- 병렬 샘플링이나 빔 서치(Beam search) 같은 복잡한 디코딩 기법을 사용할 때, 메모리가 연속되어야 한다는 제약 때문에 요청 간 물리적 메모리를 유연하게 공유하지 못하고 데이터를 중복 저장해야 함
- 결과적으로 이러한 비효율적인 메모리 관리로 인해 한 번에 처리할 수 있는 배치 크기가 제한되어 전체 시스템의 연산 자원을 다 쓰기도 전에 메모리 병목으로 처리량이 크게 떨어짐



논문의 contributions

- LLM 서빙 성능을 제한하는 주요 병목 현상이 연산 능력(Compute)이 아닌 KV 캐시 메모리의 비효율적 할당 방식에 있음을 최초로 규명함
- 운영체제의 가상 메모리 페이징(Paging) 기술에서 영감을 받아, KV 캐시를 고정된 크기의 블록(Page)으로 나누어 비연속적인 물리 메모리에 저장할 수 있게 하는 **PagedAttention** 알고리즘을 제안함
- PagedAttention을 기반으로 한 LLM 서빙 엔진인 **vLLM**을 개발하여 KV 캐시 메모리 낭비(단편화)를 거의 0(near-zero) 수준으로 근접함
- 블록 단위 관리를 통해 요청 내부 및 요청 간의 KV 캐시 메모리 공유를 가능하게 하여 메모리 사용량을 추가로 대폭 절감함
- 기존 최고 수준의 시스템(FasterTransformer, Orca)과 비교하여 동일 지연 시간 (Latency) 조건에서 전체 처리량(Throughput)을 2~4배 향상시켰으며, 시퀀스가 길고 디코딩 방식이 복잡할수록 그 효과가 더욱 뚜렷함을 입증함

2. Related Work

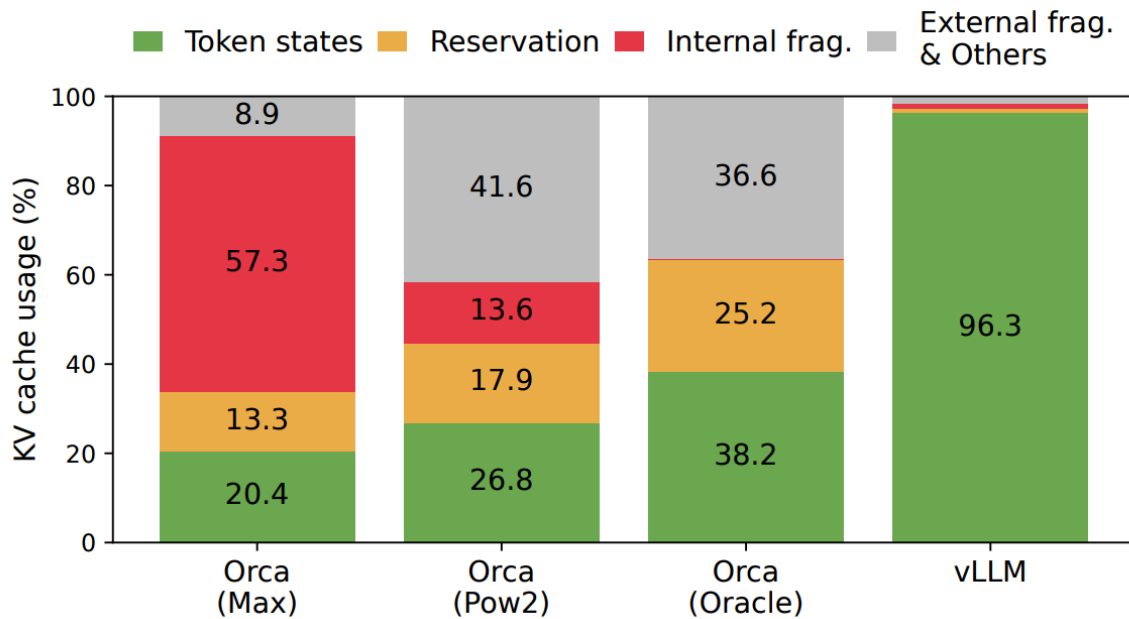
1) Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가

- **LLM 서빙 시스템:** Orca, FasterTransformer, FlexFlow-Serve 등 기존 시스템들이 처리량 향상을 위해 배치(Batch) 처리와 스케줄링 최적화에 집중해 왔음을 언급함
- **Orca의 한계:** Orca가 '반복 수준 스케줄링(Iteration-level scheduling)'을 도입해 효율을 높였으나, KV 캐시 관리에 있어서는 여전히 각 요청에 대해 연속적인 메모리 공간

을 할당하는 방식을 사용한다고 지적함

- **기존 메모리 관리의 문제:** 기존 시스템들이 요청의 최대 길이에 맞춰 메모리를 미리 확보하기 때문에, 실제 사용되지 않는 공간이 낭비되는 단편화 문제를 해결하지 못하고 있음을 강조함
- **기존 어텐션 최적화:** FlashAttention 등 기존의 어텐션 최적화 연구들은 주로 연산 속도나 I/O 효율에 집중했을 뿐, 서버 환경에서의 동적인 메모리 관리 문제는 다루지 않았다고 서술함

2) 제안 방법의 차별성을 어떻게 표현하고 있는가



- **가상 메모리 기법의 이식:** 전통적인 운영체제(OS)에서 단편화 문제를 해결하기 위해 사용하던 가상 메모리와 페이징(Paging) 개념을 LLM KV 캐시 관리에 최초로 적용함
- **비연속적 메모리 할당:** 메모리를 물리적으로 연속되게 배치해야 한다는 제약을 없애고, 고정된 크기의 블록 단위로 나누어 필요할 때마다 동적으로 할당하는 방식을 통해 내부/외부 단편화를 근절함
- **효율적인 메모리 공유:** 기존 시스템들이 병렬 샘플링 시 데이터를 중복 저장해야 했던 것과 달리, 블록 단위의 물리 메모리 매핑을 통해 요청 간 KV 캐시를 효율적으로 공유하는 '쓰기 시 복사(Copy-on-Write)' 메커니즘을 구현함
- **시스템 계층에서의 최적화:** 단순한 알고리즘 개선을 넘어, 실제 서버 엔진(vLLM) 수준에서 메모리 관리자를 직접 설계하여 실질적인 처리량(Throughput) 성능을 2~4배까지 끌어올린 점을 차별화된 성과로 내세움

3. 제안 방법론

Main Idea

- 대규모 무레이블 말뭉치로 언어 모델을 훈련하는 비지도 사전 학습 단계와, 해당 파라미터를 타겟 태스크의 레이블 데이터에 맞게 업데이트하는 지도 미세 조정 단계로 구성된 2단계 프레임워크를 제안함

$$P(x) = P(x_1) \cdot P(x_2 | x_1) \cdots P(x_n | x_1, \dots, x_{n-1}). \quad (1)$$

- 장기 의존성을 효과적으로 처리하기 위해 기존의 순환 신경망 대신 트랜스포머 디코더 아키텍처를 채택하였으며, 입력 문맥에 대해 Multi-headed self-attention 연산을 수행함
- 미세 조정 단계에서 타겟 태스크의 분류 목적 함수뿐만 아니라 기존의 언어 모델링 목적 함수를 보조로 함께 사용하여, 모델의 Generalization 성능을 높이고 수렴 속도를 가속함
- 복잡하고 다양한 형태의 타겟 태스크 입력을 처리하기 위해, 시작 토큰, 끝 토큰, 구분자 토큰을 적절히 끼워 넣어 하나의 연속된 1차원 시퀀스로 평탄화하는 입력 변환 기법을 사용함

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top, \quad (4)$$

Contribution

- 서론에서 목표로 한 '보편적인 텍스트 표현 학습'과 '최소한의 아키텍처 변경'이라는 비전이, 트랜스포머 기반의 2단계 학습법과 특수 토큰을 활용한 입력 변환 기법으로 본론에 완벽히 일치하게 전개됨
- 기존 방법론들이 타겟 태스크마다 모델의 뼈대를 대대적으로 수정해야 했던 구조적 한계를, 단순한 시퀀스 이어 붙이기(Traversal-style approach)로 어떻게 우회하고 극복했는지 논리적 근거와 구체적인 결합 방식을 통해 명확하게 입증함
- 트랜스포머 블록을 통과한 마지막 활성화 값이 선형 출력 레이어와 어떻게 수식적으로 연결되는지 명료하게 서술하였으며, 텍스트 분류, 함의, 의미 유사도, 다중 선택 등 각 문제 유형별로 데이터를 어떻게 이어 붙여야 하는지까지 아주 구체적으로 명시하여 별도의 추측 없이 바로 코드로 구현 가능하도록 친절하게 작성됨

4. 실험 및 결과

- 서론에서 강조한 'KV 캐시 메모리 낭비 근절'과 '처리량(Throughput) 혁신'을 검증하기 위해 기존 최적화 시스템인 FasterTransformer 및 Orca와 성능을 직접 비교하는 실험을 수행함
- 다양한 모델 크기(LLaMA 7B~13B, OPT 13B~175B)와 실제 사용자 데이터셋(ShareGPT, Alpaca)을 활용하여 제안 방법의 범용성과 실효성을 입증함

Dataset

- **ShareGPT**: 실제 사용자들이 챗봇과 나눈 대화를 바탕으로 한 데이터셋으로, 입력과 출력의 길이가 매우 가변적인 실제 서빙 환경을 모사함
- **Alpaca**: 고정된 형태의 요청이 들어오는 환경에서의 성능을 측정하기 위해 사용됨
- **LLM Models**: LLaMA(7B, 13B)와 초대형 모델인 OPT(13B, 66B, 175B)를 실험 대상으로 삼아 모델 규모에 따른 확장성을 검증함

Baseline

- **FasterTransformer (FT)**: NVIDIA에서 개발한 기존의 고성능 LLM 추론 엔진으로, 연속적인 메모리 할당 방식을 사용하는 대표적인 모델
- **Orca**: 요청 수준이 아닌 반복(iteration) 수준의 스케줄링을 도입하여 처리량을 개선한 기존의 최상위 서빙 시스템

결과

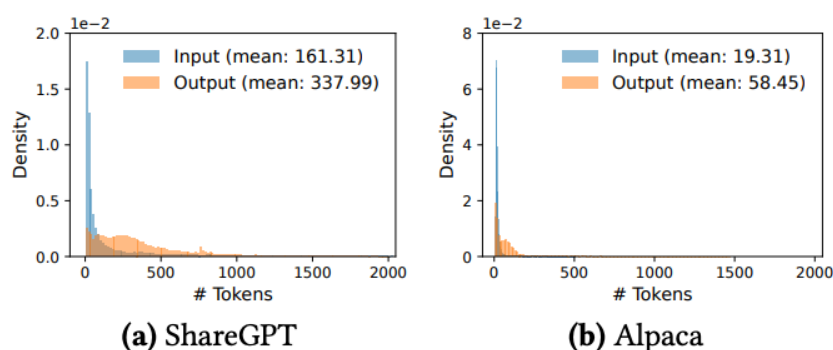


Figure 11. Input and output length distributions of the (a) ShareGPT and (b) Alpaca datasets.

- **압도적인 처리량 향상**: 동일한 지연 시간(Latency) 조건에서 FasterTransformer 대비 최대 4배, Orca 대비 최대 2.2배의 처리량 향상을 달성함

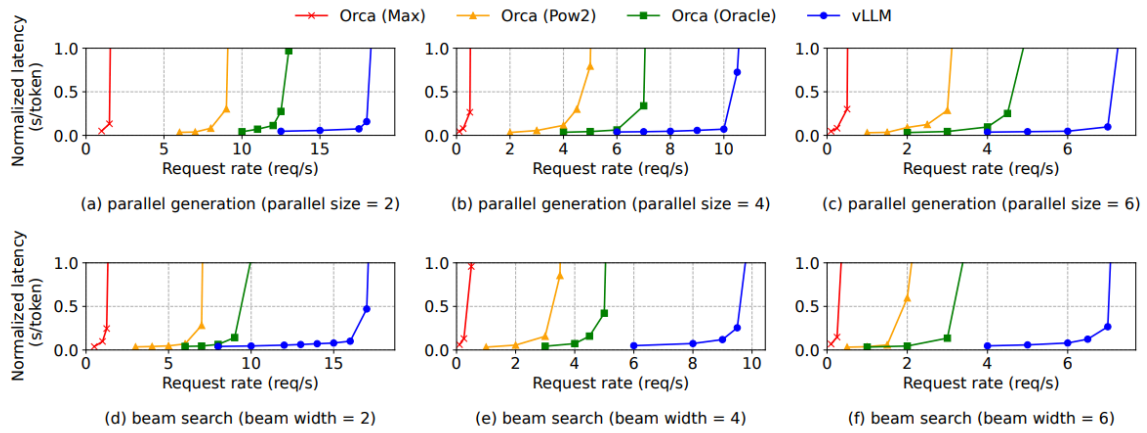
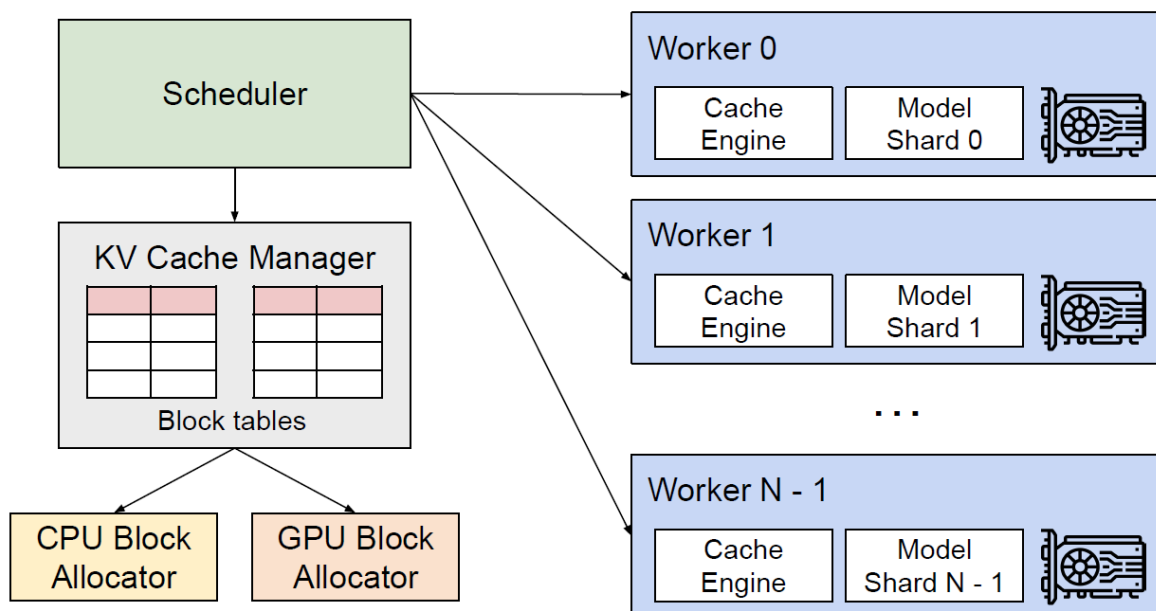


Figure 14. Parallel generation and beam search with OPT-13B on the Alpaca dataset.

- **메모리 효율의 극대화:** PagedAttention을 통해 메모리 낭비를 줄인 결과, 기존 시스템 보다 훨씬 더 큰 배치 크기(Batch size)를 수용할 수 있게 되어 하드웨어 활용률을 최대 로 끌어올림
- **복잡한 디코딩에서의 강점:** 병렬 샘플링(Parallel sampling)이나 빔 서치(Beam search) 사용 시, 공유 블록을 통해 메모리 사용량을 최대 55%까지 추가 절감하며 성능 우위를 확고히 함
- **긴 시퀀스 처리 능력:** 문장이 길어질수록 메모리 관리 효율의 차이가 극명해지며, 시퀀스 길이가 길어지는 환경에서 vLLM의 성능 이점이 더욱 커짐을 확인함

5. 결론 (배운점)

1) 연구의 의의 및 한계점



- **연구의 의의:** LLM 서빙의 핵심 병목 현상이 GPU의 연산량(Compute)이 아닌 'KV 캐시 메모리의 비효율적 관리'에 있음을 정확히 짚어냄.
 - 운영체제(OS)의 가상 메모리 Paging 기법을 AI 추론 시스템에 성공적으로 이식하여, 메모리 단편화를 제거하고 처리량을 2~4배 향상시킴으로써 대규모 AI 모델의 상용화 및 서비스 인프라 비용 절감에 막대한 기여를 함
- **연구의 한계점:**
 - 메모리 대역폭보다 연산 자체가 더 큰 병목이 되는 매우 작은 배치 크기나 짧은 시퀀스 처리 환경에서는, PagedAttention의 메모리 매핑 연산 등 추가적인 커널 오버헤드로 인해 기존 연속 할당 방식 대비 속도 향상이 미미하거나 오히려 성능이 소폭 하락할 수 있음
 - 프롬프트 간 Prefix sharing 기능은 토큰 시퀀스가 완전히 정확하게 일치할 때만 작동하므로, 의미는 같으나 단어 구성이 미세하게 다른 프롬프트들 사이에서는 캐시를 공유하지 못한다는 구조적 한계가 존재함

이번 논문은 학부 운영체제 수업에서 다루는 고전적인 '가상 메모리' 개념을 LLM의 최신 과제인 KV 캐시 문제에 접목해 냈다는 점에서 발상의 전환이 돋보여 무척 흥미로웠다. 아이디어 자체는 직관적이고 명쾌하지만, 이를 실제 GPU에서 병렬로 동작하도록 CUDA 커널 레벨로 구현하는 것은 메모리 계층에 대한 깊은 이해가 필요해 학생 수준에서 직접 바닥부터 코드를 짜보기엔 진입장벽이 상당히 높게 느껴지기도 했다. 하지만 제한된 하드웨어 자원 위에서 모델을 얼마나 경제적으로 서빙해낼 수 있는지가 실제 산업계의 진짜 핵심 과제를 뼈저리게 체감할 수 있었다. 특히 최첨단 AI 시스템의 병목을 해결하는 열쇠가 다른 아닌 OS와 같은 컴퓨터공학의 기초에서 나온다는 사실을 보며, 특정 딥러닝 알고리즘에만 치우치지 않고 시스템 전반을 아우르는 탄탄한 기본기를 다져야겠다는 다짐을 하게 되었다.

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

1. Introduction

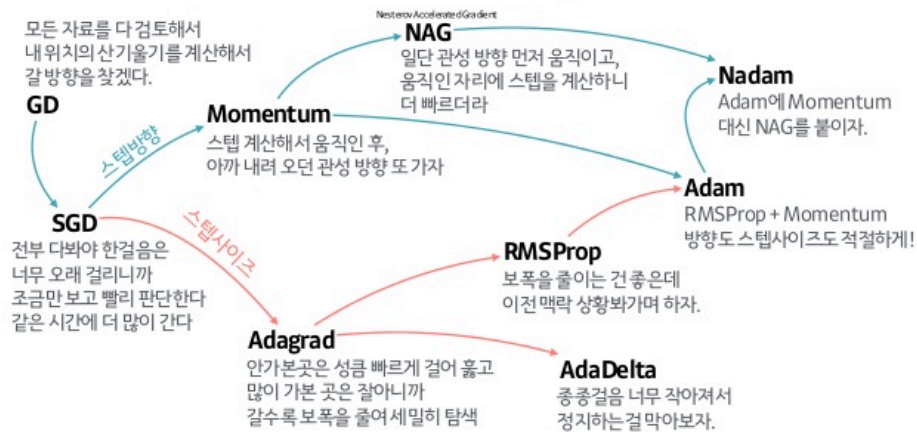
논문이 다루는 분야

- 확률적 목적 함수(Stochastic objective functions)를 최적화하기 위한 1차 미분 (First-order gradient) 기반의 최적화 알고리즘 분야
- 데이터와 파라미터의 규모가 매우 큰 대규모 머신러닝 및 딥러닝 모델 학습 과정에서, 파라미터를 효율적이고 안정적으로 업데이트하는 방법론 연구

해당 task에서 기존 연구 한계점

- 방대한 데이터와 복잡한 신경망을 다루기 위해 확률적 그래디언트 기반 최적화 (Stochastic gradient-based optimization)가 필수적이거나, 기존 방법들은 노이즈가 많거나 Sparse gradient 환경에서 안정성이 떨어짐
- 기존 알고리즘인 AdaGrad는 희소한 그래디언트를 다루는 데는 탁월하지만, Non-stationary 목적 함수에서는 성능이 크게 저하되는 한계가 있음
- 또 다른 알고리즘인 RMSProp은 비정상 목적 함수 문제를 해결하여 훌륭한 성능을 보였으나, 그래디언트의 1차 모멘트(평균)를 함께 적응적으로 활용하는 기능은 부족함
- 하이퍼파라미터 튜닝에 대한 의존도가 높고, 연산 효율성과 메모리 절약을 동시에 달성하면서도 빠르게 수렴하는 범용적인 통합 최적화 알고리즘이 부재함

산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



논문의 contributions

- AdaGrad와 RMSProp의 장점만을 성공적으로 결합하여, 그래디언트의 1차 모멘트(평균)와 2차 모멘트(비중심 분산)의 추정치를 모두 적응적으로 계산하는 최적화 알고리즘 'Adam'을 제안함
- 그래디언트의 Diagonal rescaling에 영향을 받지 않아 스케일 변화에 강건하며, 메모리 요구량이 적고 연산 효율성이 매우 뛰어나다는 것을 입증함
- 하이퍼파라미터들이 직관적인 의미를 가지며, 기본값만으로도 대부분의 환경에서 추가적인 튜닝 없이 곧바로 우수한 성능을 냄
- Online convex optimization 프레임워크를 바탕으로, 기존 최고 수준의 결과들과 비교되는 수렴 속도에 대한 수학적 증명을 제공함
- 다양한 딥러닝 모델과 데이터셋을 활용한 실험을 통해 다른 확률적 최적화 방법론들보다 실제 환경에서 압도적으로 우수하게 작동함을 실증함

- Infinity norm을 기반으로 한 Adam의 변형 알고리즘인 'AdaMax'를 추가로 고안하여 제시함

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0 : Initial parameter vector
 $m_0 \leftarrow 0$ (Initialize 1st moment vector)
 $v_0 \leftarrow 0$ (Initialize 2nd moment vector)
 $t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
 $t \leftarrow t + 1$
 $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
 $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
 $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
 $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
 $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
 $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)
